Legacy Modernization Platform — Multi-Agent Architecture Design

Multi-tenant platform for documenting the As-Is architecture of government clients' legacy systems (ArchiMate primary, UML/C4 derived), assessing them across 133 maturity dimensions, defining the gap to a professionally-authored Target architecture, and planning a modernization roadmap. Built on the Deep Agents framework (LangChain/LangGraph).

Status: architecture design, pre-implementation. Requirements gathered through iterative clarification — see "Open items" at the end for what's still pending (rubric Excel sheet, model-hosting choice per client, on-prem ops model).

---

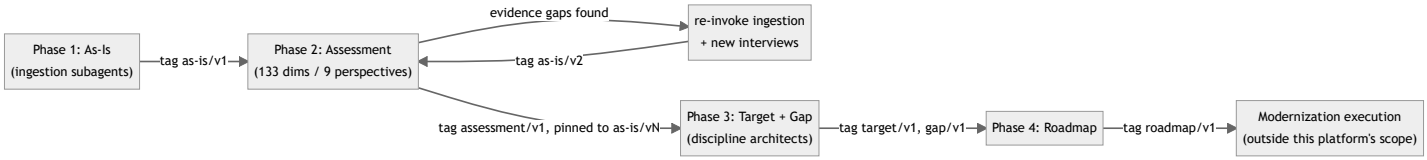
1. Design principles

1. **ArchiMate is the source of truth.** UML and C4 are projections of the same underlying model, not independently maintained artifacts.
2. **Everything sensitive is tenant-isolated at the infrastructure level**, not just the query level — silo DB per tenant, tenant resolved only from an authenticated claim, never from client-supplied input.
3. **Every fact is traceable.** Every model element carries evidence citations; every artifact version links back to the agent run that produced it and the human approval that accepted it.
4. **Ingestion is a shared service, not a phase-1 artifact.** The same subagents that build the As-Is baseline are re-invoked during Assessment to close evidence gaps.
5. **Four infrastructure concerns are pluggable per tenant**, because deployment mode (cloud vs. fully on-prem) and data-sovereignty needs vary per client: **git hosting, object storage, LLM provider, observability backend.** Same adapter pattern for all four.
6. **Strategic outputs (Target architecture, Roadmap) are agent-drafted, human-decided.** Everything else defaults to lighter-touch review.

---

2. Phase flow and versioning

Each phase's output is a tagged commit in the tenant's model repo. Every downstream phase pins to an exact tag — never "current state" — so a Gap analysis is always reproducibly tied to one As-Is snapshot and one Assessment run.



Every arrow that crosses a phase boundary is a **human approval gate** (PR merge in the tenant's git repo).

---

3. Subagent catalog

3.1 Shared services (invoked by multiple phases)

Subagent	Role	Invoked by
`strategy-analyst`	Extracts Motivation + Strategy layer elements from strategic plans, policy/compliance docs, business case, SME interviews	Phase 1, re-invoked in Phase 2
`business-analyst`	Extracts Business layer (Actor, Role, Process, Function, Service) from docs, interviews	Phase 1, re-invoked in Phase 2
`code-analyzer`	Extracts Application layer from repos, DB schemas	Phase 1, re-invoked in Phase 2

Subagent	Role	Invoked by
`infra-analyzer`	Extracts Technology & Infrastructure layer from IaC, CMDB, network configs	Phase 1, re-invoked in Phase 2
`integration-mapper`	Extracts cross-layer and **cross-system** relationships from API specs, message queues, traffic data	Phase 1, re-invoked in Phase 2
`reconciler`	Dedupes/merges fragments from multiple ingestion subagents into the canonical registry (deterministic fuzzy-match proposes candidates, LLM confirms merge-vs-keep-separate)	All phases that write to the model
`validator`	Checks every element/relationship against the ArchiMate metamodel skill and confirms evidence citations exist; produces coverage/gap reports	All phases
`view-generator`	Projects the canonical ArchiMate model into C4 (Context/Container/Component) and UML (Component/Deployment/Class) views, per the documented projection-mapping skill; LLM judgment reserved for C4 zoom-level curation only	Phase 1 (and on-demand refresh)

3.2 Phase 2 — Assessment

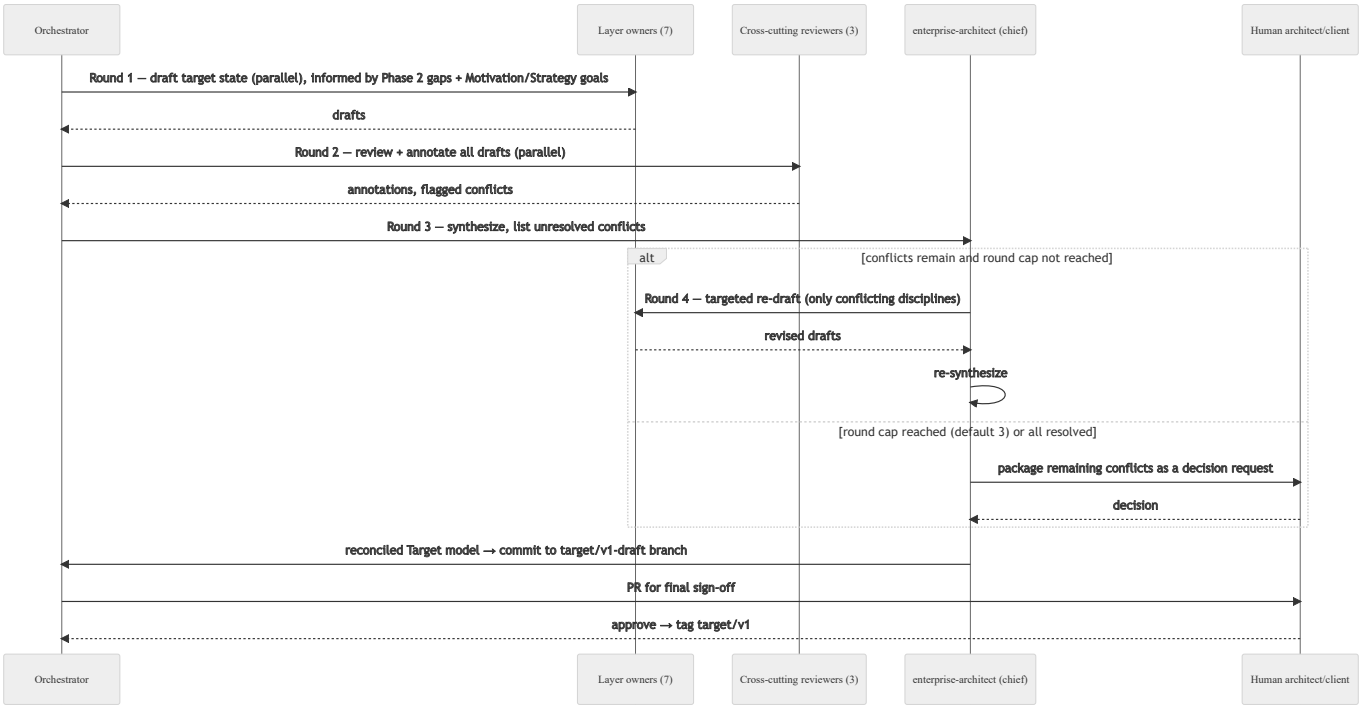
Subagent	Role
`perspective-assessor` (×9: strategy, business, application, UX, technology, infrastructure, operation, integration, security)	For each dimension in its perspective: checks evidence sufficiency against the rubric → requests targeted re-ingestion/interviews via the orchestrator if insufficient → scores 1–5 with rationale and evidence citations, using the perspective's rubric skill

3.3 Phase 3 — Target architecture & Gap

Mimics a real multi-discipline consulting team. Layer-owning architects draft; cross-cutting reviewers annotate; the chief synthesizes.

Subagent	Role	Type
`enterprise-architect` (chief)	Sets Motivation/Strategy-layer goals/principles/constraints that bound every other discipline; runs conflict synthesis across rounds; owns final coherence	Layer owner + synthesizer
`business-architect`	Business layer target state	Layer owner
`application-architect`	Application layer target state	Layer owner
`infrastructure-architect`	Technology & Infrastructure layer target state	Layer owner
`integration-architect`	Cross-system/cross-component target relationships, API and event-driven integration patterns	Layer owner
`solution-architect`	Bridges Business ↔ Application, translates capabilities into solution building blocks	Layer owner (bridging)
`ux-architect`	Application-layer interaction/interface elements + companion UX artifacts (journeys, service blueprints — outside ArchiMate's native scope, stored alongside)	Layer owner (bridging)
`security-architect`	Reviews every layer owner's draft; injects security requirements/constraints	Cross-cutting reviewer
`data-architect`	Reviews for data ownership, flow, governance across Business/Application/Technology	Cross-cutting reviewer
`devsecops-architect`	Reviews for deployability, CI/CD, observability, operational readiness	Cross-cutting reviewer
`gap-analyzer`	Diffs pinned As-Is baseline vs. synthesized Target, element-by-element and dimension-by-dimension; produces gap records with effort/complexity estimates	Downstream of synthesis

Negotiation protocol (bounded auto-negotiation, escalate on cap):



3.4 Phase 4 — Roadmap

Subagent	Role
`roadmap-planner`	Consumes gap records + cross-system integration graph (sequencing constraints) + client constraints; drafts initiatives tagged with a modernization strategy (Rehost / Replatform / Refactor / Rearchitect / Rebuild / Replace / Retain); heaviest HITL gate of the four phases

3.5 Orchestrator

One long-lived thread per phase run. Owns `TodoListMiddleware` planning, dispatches subagents via `SubAgentMiddleware`, runs the SME interview loop directly (subagents are stateless single-shot — live interviews belong in the orchestrator's own multi-turn thread), and triggers git commit/PR steps deterministically (not via LLM-authored git operations).

---

4. Skills catalog

`SkillsMiddleware`, loaded on demand — each subagent loads only what it needs.

Skill	Purpose	Used by
`archimate-metamodel`	Full ArchiMate 3.2 metamodel: element types per layer (incl. Motivation/Strategy), relationship types, and the relationship-validity matrix	All ingestion subagents, `reconciler`, `validator`, all Phase 3 architects
`archimate-projection-c4-uml`	Documented mapping rules: ArchiMate elements → C4 Context/Container/Component, → UML Component/Deployment/Class	`view-generator`
`assessment-rubric-`	The 5-level maturity rubric for that perspective's dimensions (from the client-provided Excel sheet — pending)	Corresponding `perspective-assessor`

Skill	Purpose	Used by
`<perspective>` (×9)		
`discipline-reference-<discipline>` (×10)	Reference architectures/patterns/standards for that discipline (e.g. security-architect loads gov security frameworks; infrastructure-architect loads cloud landing zone patterns; data-architect loads data governance patterns)	Corresponding Phase 3 architect
`modernization-strategy-taxonomy`	The 7 R's, decision criteria for which strategy fits which gap profile	`roadmap-planner`
`interview-guide-generation`	Turns a validator gap list into a targeted SME interview guide	Orchestrator (Phase 1 and Phase 2)

5. Tenant git repository layout

One repo per tenant (git-backed, real diff/branch/merge/PR review — the mechanism for every HITL gate in this system).

```
tenant-repo/
├── systems/
│   └── <system-id>/
│       ├── as-is/
│       │   ├── motivation/
│       │   ├── strategy/
│       │   ├── business/
│       │   ├── application/
│       │   └── technology/
│       ├── views/
│       │   ├── c4/
│       │   └── uml/
│       ├── target/
│       │   └── drafts/<discipline>/<round-n>/   # working, merged by chief into target
│       ├── gap/
│       └── evidence-index/        # citations only — large binaries live in object sto
```

```
├── landscape/
|   ├── integrations.json          # cross-system relationship graph
|   └── landscape-views/
└── roadmap/
    └── initiatives/
```

Tags: `as-is/v1`, `as-is/v2`, … · `assessment/v1` (pinned to an as-is tag) · `target/v1` · `gap/v1` · `roadmap/v1`

Branches: `feature/ingest-<system>-<run-id>`, `feature/target-<discipline>-<round>`, `target/v1-draft` (integration branch during negotiation)

Git provider adapter — one interface (`create_branch`, `commit_files`, `open_pull_request`, `get_diff`, `merge_pr`, `read_file_at_ref`), two implementations selected per tenant:

- Self-hosted git server (e.g. Gitea/Gitness) — for on-prem clients
- GitHub API — for cloud-hosted clients

---

6. Database design

6.1 Control plane (shared, low-sensitivity)

Table	Purpose
`tenants`	id, name, deployment_mode (cloud/on-prem), git_provider_config, storage_provider_config, model_provider_config, observability_provider_config
`users`, `roles`, `tenant_memberships`	RBAC: client SME/reviewer, client admin, platform/consultant admin
`dimension_catalog`	The 133 dimensions + 9 perspectives — shared methodology, not client data
`jobs`	Async agent-run tracking: tenant_id, system_id, phase, status, run_id (correlates to observability trace), timestamps
`audit_log`	Every commit, approval, and cross-tenant-boundary access attempt

6.2 Tenant silo (per client — schema-per-tenant by default, dedicated DB instance for clients whose compliance posture requires it)

Table	Purpose
`legacy_systems`	Systems owned by this tenant
`system_integrations`	Structured mirror of the cross-system relationship graph (for fast querying without parsing git)
`model_element_index`	element_id, system_id, layer, archimate_type, name, git_path, current_commit — index only; full content (evidence, docs, relationships) lives in the git JSON files
`assessment_runs` , `assessment_scores`	Scores reference `dimension_catalog` in the control plane
`target_drafts`	Per-discipline, per-round drafts during Phase 3 negotiation
`conflicts`	discipline_a, discipline_b, description, resolution_status, resolved_in_round — tracks the negotiation protocol explicitly
`gap_records`	Linked to the assessment dimension and target element that motivated each gap
`roadmap_initiatives`	Modernization-strategy tag, sequencing/dependency links
`artifact_versions`	commit sha, phase, tag, author_type (agent/human), **run_id** (join key back to the observability trace), approval_status, approved_by, approved_at

7. Provider abstraction layer

The same adapter pattern, four times, because deployment mode varies per tenant:

Concern	Cloud default	On-prem / air-gapped option
Git hosting	GitHub API	Self-hosted git server (Gitea/Gitness)

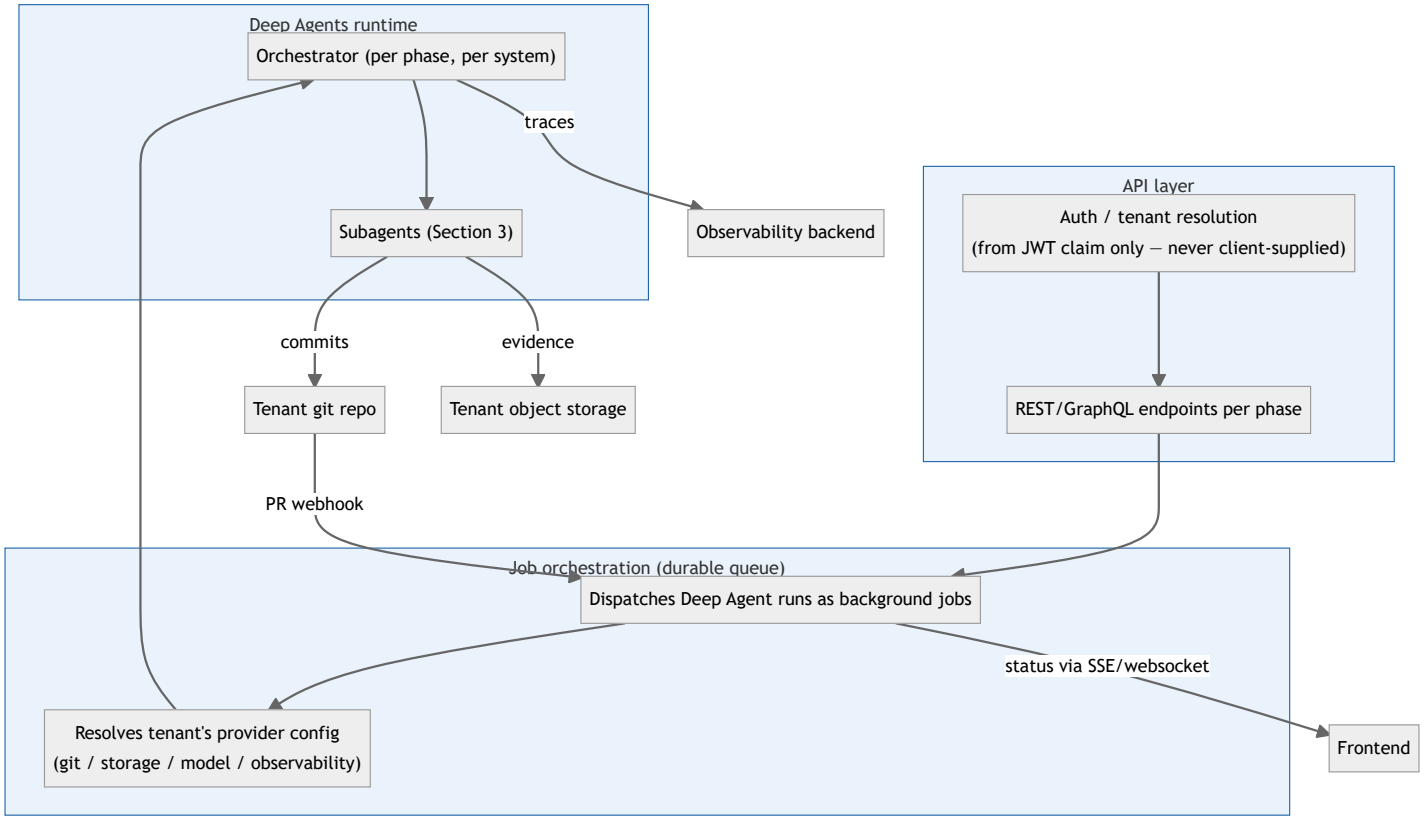
Concern	Cloud default	On-prem / air-gapped option
Object storage (evidence docs, renders)	S3-compatible cloud bucket, per-tenant prefix	MinIO or equivalent, self-hosted
LLM provider	Cloud model API (e.g. Claude)	Pluggable — self-hosted/open-weight model, or the client's own private cloud endpoint (Bedrock/Azure/Vertex private connection); **left open per client**, agent runtime built against a swappable model-provider interface from day one
Observability	LangSmith (cloud or self-hosted)	Langfuse (self-hosted) as primary open-source option; raw OTel → Grafana Tempo/Prometheus/Loki when a client already has its own stack

Selection lives in `tenants.*_provider_config`; resolved at job-dispatch time, never client-supplied.

Regardless of backend, the trace-to-artifact link (`run_id` ↔ git commit ↔ PR/approval) is bespoke — no observability tool gives you that; it's the join in `artifact_versions`.

---

8. Backend architecture



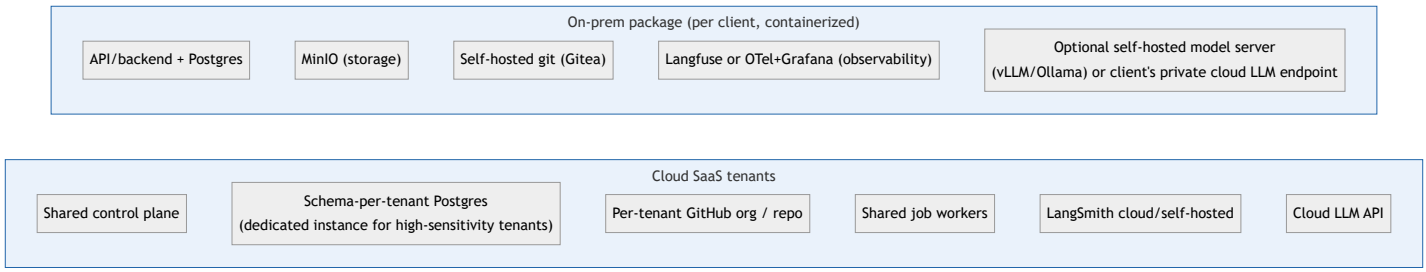
Key rules:

- Tenant resolution is structural (JWT claim → DB connection + git repo path), not request-parameter-driven — no code path can cross tenant boundaries by mistake.
- HITL gates resolve via git PR review webhooks, not bespoke approval UI — reuse the git host's review tooling.
- Every subagent invocation tags `{tenant_id, system_id, phase, run_id}` for both audit and cost/token attribution.

9. Frontend modules

Module	Purpose
Tenant workspace	System switcher (multi-system tenants), phase navigator mirroring git tags
Model viewer	Layered ArchiMate views + derived C4/UML, element detail panel with evidence citations
Diff / PR review	Git diff between tags/commits, doubling as the HITL approval surface
Assessment dashboard	133-dimension scorecard by 9 perspectives, heatmap/radar, drill-down to rationale + evidence
Target negotiation viewer	Per-discipline drafts, flagged conflicts, round history, escalation decisions — new surface specific to Phase 3's negotiation protocol
Gap register	Linked to assessment dimensions and target elements
Roadmap timeline	Dependency-aware, initiatives tagged by modernization strategy, constrained by `system_integrations`
Admin: tenant provider config	Choose git/storage/model/observability backend + credentials per tenant

10. Deployment topology



On-prem is a full deployable unit (backend, DB, agents, git, observability) — everything behind the provider abstractions in Section 7, so the same codebase serves both topologies via config, not a fork.

---

11. Suggested build sequence (MVP-first)

Given the scope, build in this order rather than all at once:

1. **Control plane + single-tenant Phase 1 (As-Is)** — ingestion subagents, ArchiMate metamodel skill, git-backed versioning, basic model viewer. Proves the core loop (evidence → model → traceable commit → PR review) before adding complexity.
2. **Multi-tenancy + provider abstractions** — schema-per-tenant, git/storage adapters (start with one backend each, e.g. GitHub + S3), tenant resolution middleware.
3. **UML/C4 projection** (`view-generator`) — validate the derived-views approach before assessment/target complexity stacks on top.
4. **Phase 2 (Assessment)** — once the rubric Excel sheet is available; reuses Phase 1 ingestion subagents, so incremental cost is mostly the 9 `perspective-assessor` subagents and skills.
5. **Observability** — wire in LangSmith or Langfuse before Phase 3, since the multi-round negotiation protocol is exactly the kind of thing you'll want trace visibility into while tuning it.
6. **Phase 3 (Target + Gap)** — highest agent-orchestration complexity; build and tune the negotiation protocol against real Phase 1/2 output.
7. **Phase 4 (Roadmap)** — depends on stable Gap output.
8. **On-prem packaging** — once the cloud path is proven, containerize and validate the provider-abstraction swap end-to-end against one pilot on-prem client.

12. Open items (pending your input)

1. **Rubric Excel sheet** — will determine the final skill granularity (per-perspective vs. per-dimension) for `assessment-rubric-*`.
2. **Model-hosting choice per client** — deferred by design; resolved per tenant at deployment time via the model-provider abstraction.
3. **On-prem operations model** — whether your team remotely operates each on-prem instance or hands off to client IT after install; affects packaging/update-delivery design but not the core architecture above.